ព្រះរាជាណាចក្រកម្ពុជា
ជាតិ សាសនា ព្រះមហាក្សត្រ

វិទ្យាស្ថានបច្ចេកវិទ្យាកម្ពុជា
ដេប៉ាតឺម៉ង់ថ្នាក់ឆ្នាំសិក្សាមូលដ្ឋាន

របាយការណ៍កិច្ចការផ្ទះ.....OOP Report.....
ប្រធានបទ : Object-Oriented Programming
ក្រុម I3-AMS1A
ធ្វើរបាយការណ៍ ថ្ងៃទី ២៦ ខែ តុលា ឆ្នាំ ២០២៦

ឈ្មោះនិស្សិត	លេខសំគាល់ខ្លួននិស្សិត	ពិន្ទុ
1. CHHON KOKHOUR	e20240556	…….

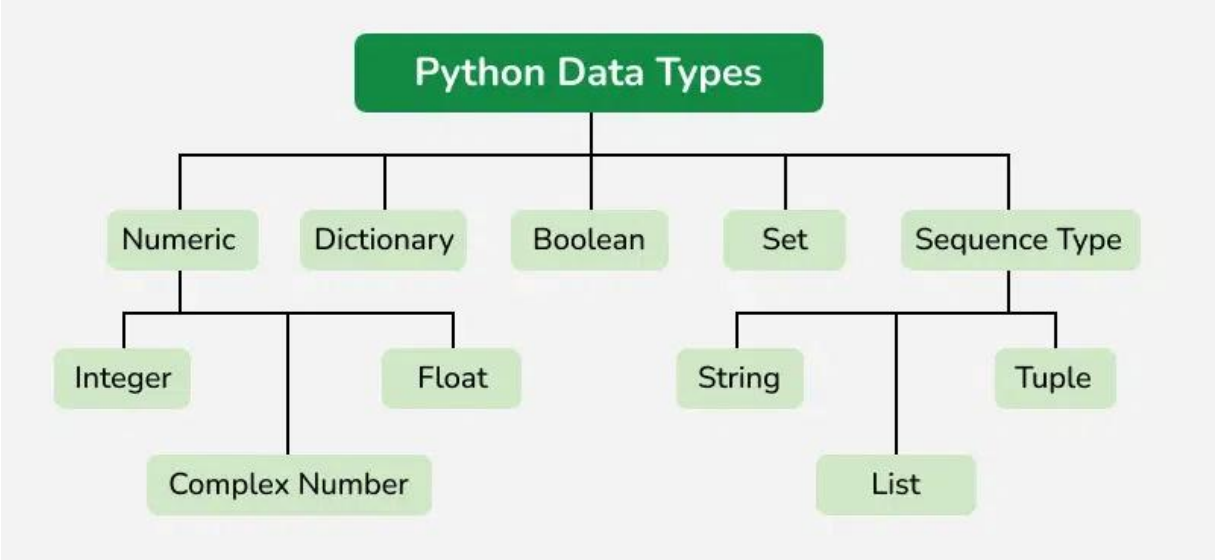
ឈ្មោះគ្រូទទួលបន្ទុកពិសោធន៍: វ៉ាន់ វិសាល

ឆ្នាំសិក្សា ២០២៥-២០២៦

Chapter 1: Python Basic Data Types

I. Definition and Explanation

a. Primitative: A data type tells Python what kind of value a variable holds and which operations are allowed.



b. Collection and Reference Types (List, Tuple, Set, and Dictionary in Python)

❖ **List**: is an ordered, mutable collection enclosed in square brackets []. Lists allow duplicate elements and support indexing, slicing, and modification operations.

➢ Key Features

- **Mutable:** You can add, remove, or modify elements after creation
- **Ordered:** Elements maintain their insertion order

- **Duplicates allowed:** Same values can appear multiple times
- **Indexing and slicing:** Access elements by position

❖ **Tuple**: is an ordered, immutable collection enclosed in parentheses (). Once created, you cannot modify its elements, making tuples ideal for storing fixed data.

➢ Key Features

- **Immutable:** Cannot change elements after creation
- **Ordered:** Elements maintain their insertion order
- **Duplicates allowed:** Same values can appear multiple times
- **Memory efficient:** Uses less memory than lists

❖ **Dictionary**: is a mutable collection of key-value pairs enclosed in curly braces {}. Dictionaries provide fast lookup based on unique keys.

➢ Key Features

- **Mutable:** Can add, remove, or modify key-value pairs
- **Ordered:** Maintains insertion order (Python 3.7+)
- **Unique keys:** Keys must be unique and immutable
- **Key-based access:** Access values using keys

❖ **Set**: an unordered collection of unique elements enclosed in curly braces {}. Sets automatically

eliminate duplicates and support mathematical set operations.

- Key Features
  - **Mutable:** Can add or remove elements
  - **Unordered:** No guaranteed order of elements
  - **No duplicates:** Automatically removes duplicate values
  - **Set operations:** Union, intersection, difference

II. Java and Python concept

Concept	**Java**	**Python**
Declare an int	int age = 20;	Age =20
Text	String name ="Dara";	Name ="Sophea"
Single character	Char c ='A';	C ='A'
Boolean value	True / false	True / False
No value	null	None

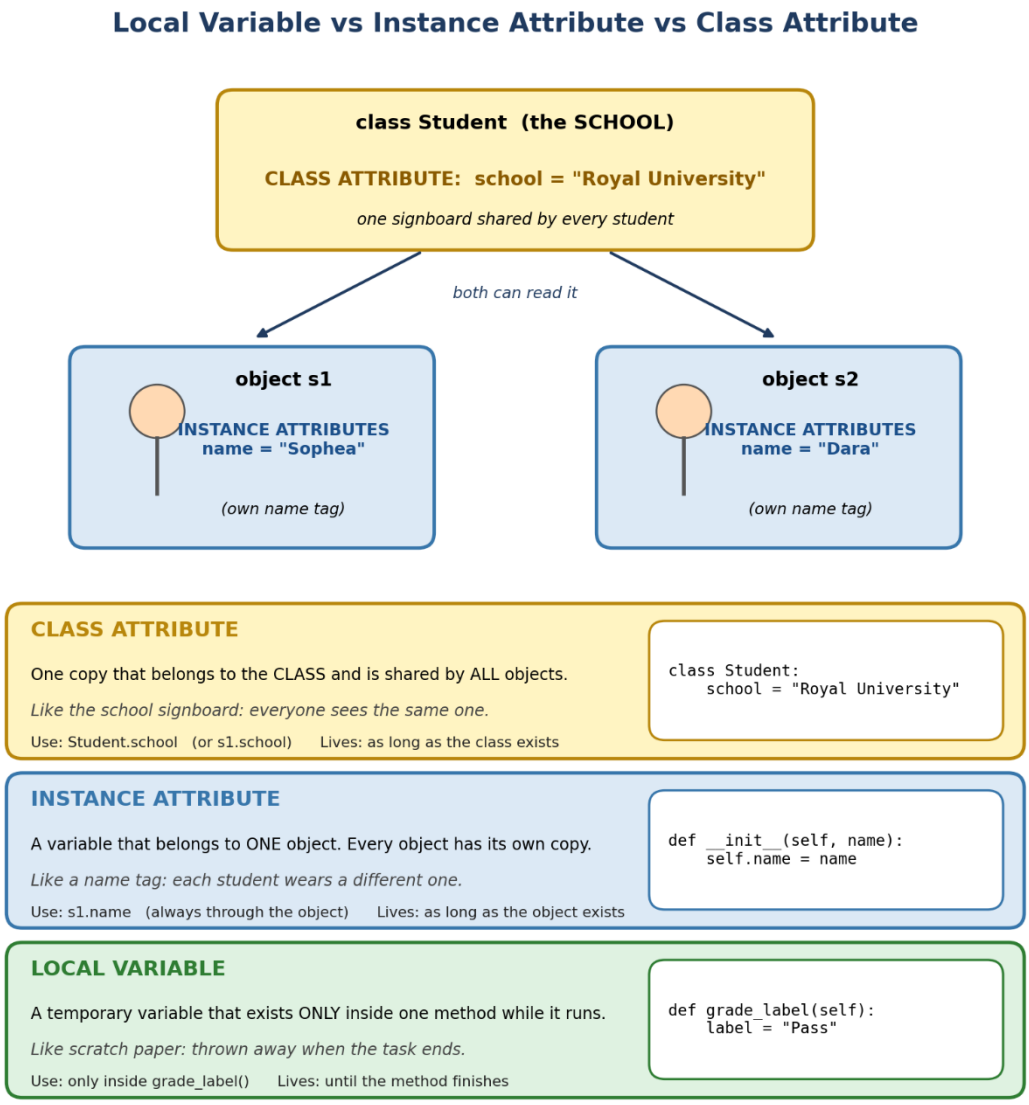
Chapter 2: Python variables and Attributes

I. Definition of Variables and Attribute

a. Local Variables: A local variable is created inside a function or method and exists only while that function runs.

b. Instance Attributes: An instance attribute is a variable that belongs to one specific object. It is normally created in **__init__** with **self.variable = value.**

c. Class Attributes: A class attribute is a variable defined directly in the class body, outside every method. It belongs to the class and is shared by all instances.



II. Example: The student class

```
OOP(tp).py > codeslide.py > Student
1   class Student:
2       school = "ITC"  # Class Attribute
3
4       def __init__(self, name):
5           self.name = name  # Instance Attribute
6
7       def hello(self):
8           message = "Hello"  # Local Variable
9           print(message, self.name, self.school)
10
11
12  s1 = Student("Dara")
13  s2 = Student("Sokha")
14
15  s1.hello()
16  s2.hello()
```

➢ Output

```
Hello Dara ITC
Hello Sokha ITC
```

- **Local** = temporary inside a method. Instance = belongs to one object (self.x). Class = shared by all objects (defined in the class body).
- **Watch out:** to change a class attribute, use the class (Student.school_name = ...). Assigning through an instance only creates a new instance attribute.

Chapter 3: Python Modifiers and Access Control

I. Definition and Explanation

a. **Public, Non-Public and Private Naming**: Python has no public, protected or private keywords. Instead the name signals the intended access level: name is public, **_name** is non-public (internal use) and **__name** is private.

Naming convention	**Example**	**Intended access**
Public	self.name	Can be accessed from anywhere
Non-public (internal)	self.name	Intended for internal use
Private (name-mangled)	self._name	Intended to prevent accidental access or overriding

b. **Access Control by Convention:** the process where Python renames an attribute that starts with two underscores (and does not end with two) to _ClassName__attr.

c. **@Staticmethod() function:** is used to convert a function to a static function. Static methods are independent of class instances, meaning they can be called on the class itself without requiring an object of the class.

d. **@classmethod**: is used to create a method that works with the class instead of an object instance. A class method receives the class itself as the first argument using cls. It is commonly used to access class variables, create factory methods and perform operations related to the class.

II. Practical example: Controlling access to a BankAccount class

```python
class BankAccount:
    bank_name = "ABC Bank"
    total_accounts = 0

    def __init__(self, owner, initial_balance=0):
        self.owner = owner
        self._account_type = "Savings"
        self.__balance = initial_balance
        BankAccount.total_accounts += 1

    # Display Information

    def display_info(self):
        print("\n--- Bank Account Information ---")
        print(f"Owner: {self.owner}")
        print(f"Bank: {BankAccount.bank_name}")
        print(f"Account Type: {self._account_type}")
        print(f"Balance: ${self.__balance:.2f}")

    # Deposit amount
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Successfully deposited ${amount:.2f}.")
        else:
            print("Deposit amount must be positive.")

    # Withdraw amount
    def withdraw(self, amount):
        if amount <= 0:
            print("Withdrawal amount must be positive.")
        elif amount > self.__balance:
            print("Insufficient funds.")
        else:
            self.__balance -= amount
            print(f"Successfully withdrew ${amount:.2f}.")
```

```python
def get_balance(self):
        return self.__balance
    #classmethod

    @classmethod
    def change_bank_name(cls, new_name):
        cls.bank_name = new_name
        print(f"Bank name changed to {cls.bank_name}.")

    @classmethod
    def display_total_accounts(cls):
        print(f"Total accounts created: {cls.total_accounts}")

# Create Two Account

account1 = BankAccount("Dara", 1000)
account2 = BankAccount("Sokha", 500)

account1.display_info()
account2.display_info()

account1.deposit(200)
account1.withdraw(150)

print(f"\nDara's current balance: ${account1.get_balance():.2f}")

print("\n--- Access Control Demonstration ---")
print("Public attribute:", account1.owner)
print("Non-public attribute:", account1._account_type)

print("\n--- Class-Level Demonstration ---")
BankAccount.display_total_accounts()

BankAccount.change_bank_name("Cambodia Bank")

print("Account 1 bank:", account1.bank_name)
print("Account 2 bank:", account2.bank_name)

print("\n--- Final Account Balances ---")
print(f"Dara: ${account1.get_balance():.2f}")
print(f"Sokha: ${account2.get_balance():.2f}")
```

➢ Output

```
--- Bank Account Information ---
Owner: Dara
Bank: ABC Bank
Account Type: Savings
Balance: $1000.00

--- Bank Account Information ---
Owner: Sokha
Bank: ABC Bank
Account Type: Savings
Balance: $500.00
Successfully deposited $200.00.
Successfully withdrew $150.00.

Dara's current balance: $1050.00

--- Access Control Demonstration ---
Public attribute: Dara
Non-public attribute: Savings

--- Class-Level Demonstration ---
Total accounts created: 2
Bank name changed to Cambodia Bank.
Account 1 bank: Cambodia Bank
Account 2 bank: Cambodia Bank

--- Final Account Balances ---
Dara: $1050.00
Sokha: $500.00
```

Chapter 4: Python Modifiers and OOP Features — Part 2

I. Definition and Explanation

a. **Abstract Classes and Methods (abc, @abstractmethod):** is a class that cannot be instantiated on its own and is designed to be a blueprint for other classes. Abstract classes allow us to define methods that must be implemented by subclasses, ensuring a consistent interface while still allowing the subclasses to provide specific implementations.

b. **Encapsulation:** means bundling data and the methods that use it inside one class, and controlling how the data is reached. It helps in achieving abstraction. A class can hide the implementation part and discloses only the functionalities required by other classes which allows later changes to representations or implementations without impacting the codes that uses this class.

c. Other OOP Features: @property, @final and typing.final:

- **@property() function**: is a built-in function that returns an object of the property class. It allows developers to create properties within a class, providing a way to control access to an attribute by defining getter, setter and deleter methods. This enhances encapsulation and ensures better control over class attributes.
- **@final**: tells Python type checkers that a class should not be inherited from or a method should not be overridden. However, it does not enforce these restrictions at runtime.

II. University Management System Example

```python
from abc import ABC, abstractmethod
from typing import final


class UniversityMember(ABC):

    university_name = "Institute of Technology of Cambodia"
    def __init__(self, name, age):
        self.name = name
        self._age = age

    #Property

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("Age cannot be negative.")
        self._age = value

    #abtract,ethod

    @abstractmethod
    def display_role(self):
        pass

    #final : dosplayinfo

    @final
    def display_info(self):
        print("\n--- University Member Information ---")
        print(f"University: {self.university_name}")
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        self.display_role()
```

```python
#Student class

class Student(UniversityMember):

    def __init__(self, name, age, student_id, major):
        super().__init__(name, age)
        self.student_id = student_id
        self.major = major

    def display_role(self):
        print("Role: Student")
        print(f"Student ID: {self.student_id}")
        print(f"Major: {self.major}")

#Teacher class

class Teacher(UniversityMember):

    def __init__(self, name, age, teacher_id, subject):
        super().__init__(name, age)
        self.teacher_id = teacher_id
        self.subject = subject

    def display_role(self):
        print("Role: Teacher")
        print(f"Teacher ID: {self.teacher_id}")
        print(f"Subject: {self.subject}")

# Cretae one student and teacher

student1 = Student("Dara", 20, "ST001", "Applied Mathematics")
teacher1 = Teacher("Sokha", 35, "TC001", "Statistics")

student1.display_info()
teacher1.display_info()

print("\n--- Updating Student Age ---")
student1.age = 21
print(f"Updated age: {student1.age}")
```

- Output

```
--- University Member Information ---
University: Institute of Technology of Cambodia
Name: Dara
Age: 20
Role: Student
Student ID: ST001
Major: Applied Mathematics

--- University Member Information ---
University: Institute of Technology of Cambodia
Name: Sokha
Age: 35
Role: Teacher
Teacher ID: TC001
Subject: Statistics
```

Chapter 5: Python Operators - Part 1

Operators tell Python how to calculate, compare values, work with bits, and combine conditions.

I. Definition and Explanation

a. **Arithmetic Operators** (+ - * / // % **): perform mathematical calculations: add, subtract, multiply, divide, floor-divide, remainder and power.

b. **Comparison Operators** (== != > < >= <=): allow you to compare two values and return a Boolean result: either True or False. Python supports comparison across different data types, such as numbers, strings and booleans. For strings, the comparison is based on lexicographic (alphabetical) order.

➢ Arithmetic & Comparison Operators (EXAMPLE)

```python
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponentiation:", a ** b)
```

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulus: 1
Exponentiation: 1000
```

```python
a = 10
b = 5

print("Equal:", a == b)
print("Not equal:", a != b)
print("Greater than:", a > b)
print("Less than:", a < b)
print("Greater than or equal:", a >= b)
print("Less than or equal:", a <= b)
```

```
C:\Users\MSI PC\OneDrive\Desk
Equal: False
Not equal: True
Greater than: True
Less than: False
Greater than or equal: True
Less than or equal: False
```

c. **Bitwise Operators** (& | ^ ~ << >>): used to perform bitwise calculations on integers. The integers are first converted into binary and then operations are performed on each bit or corresponding pair of bits and the result is then returned in decimal format.

d. **Logical Operators (and or not)**: used to combine or modify conditions and return a Boolean result (True or False). They are commonly used in conditional statements to control the flow of a program based on multiple logical conditions.

➢ Bitwise Operators and Logical Operators (EXAMPLE)

```python
a = 5
b = 3

print("Bitwise AND:", a & b)
print("Bitwise OR:", a | b)
print("Bitwise XOR:", a ^ b)
print("Bitwise NOT:", ~a)
print("Left Shift:", a << 1)
print("Right Shift:", a >> 1)
```

```python
age = 19
score = 85

print("\n=== Logical Operators ===")
print("AND:", age >= 18 and score >= 80)
print("OR:", age >= 18 or score >= 90)
print("NOT:", not age >= 18)
```

```
(.venv) C:\Users\MSI PC\OneD
Bitwise AND: 1
Bitwise OR: 7
Bitwise XOR: 6
Bitwise NOT: -6
Left Shift: 10
Right Shift: 2
```

```
=== Logical Operators ===
AND: True
OR: True
NOT: False
```

II. Practical Example: Student Grades & Eligibility

```
math = 18
physics= 35
english =42
attendance =90
total_score = math + physics +english
average = total_score/3
passed =average>=50
eligible = average>=80 and attendance>=75

print("total_score =",total_score)
print("Average:", average)
if average >= 90:
    print("Grade: A")
elif average >= 80:
    print("Grade: B")
elif average >= 70:
    print("Grade: C")
elif average >= 60:
    print("Grade: D")
elif average >= 50:
    print("Grade: E")
else:
    print("Grade: F")
print("Exam result:", "PASS" if passed else "FAIL")
print("Award eligibility:", "ELIGIBLE" if eligible else "NOT ELIGIBLE")
```

- Output

```
total_score = 95
Average: 31.666666666666668
Grade: F
Exam result: FAIL
Award eligibility: NOT ELIGIBLE
```

Chapter 6: Python Operators - Part 2

I. Definition and Explanation

a. **Assignment Operators**: used to assign values to variables and update them using different operations. These operators assign the result of an expression to the variable on the left-hand side, making variable manipulation simpler and more efficient.

```
1  x = 10
2
3  print(x == 10)
4  x += 5
5  print(x)
```

```
True
15
```

b. Special Operators

- **Identity Operators:** used to compare the objects if both objects are actually of same data type and share same memory location. There are different identity operators such as: **IS Operator** and **IS NOT Operator**

```
1  a = [10, 20, 30]
2  b = a
3  c = [10, 20, 30]
4
5  print("a is b:", a is b)
6  print("a is c:", a is c)
7  print("a is not c:", a is not c)
8
9  # Compare values
10 print("a == c:", a == c)
11
12 # Check None
13 advisor = None
14 print("No advisor assigned:", advisor is None)
```

```
(.venv) C:\Users\MSI PC\OneD
a is b: True
a is c: False
a is not c: True
a == c: True
No advisor assigned: True
```

- **Membership Operators:** is test for the membership of an object in a sequence, such as strings, lists or tuples. Python offers two membership operators to check or validate the membership of a value. They are as follows: **IN Operator ,Not IN Operator** and **operator.contains() Method.**

```python
subjects = ["Python", "Statistics", "Mathematics"]

print("Python in subjects:", "Python" in subjects)
print("English in subjects:", "English" in subjects)
print("English not in subjects:", "English" not in subjects)

# Membership in a string
university = "Institute of Technology of Cambodia"

print("Technology in university:", "Technology" in university)

# Membership in a dictionary
student = {
    "name": "Seki",
    "age": 19,
    "major": "Applied Mathematics"
}

print("name in student:", "name" in student)
print("Seki in student:", "Seki" in student)
print("Seki in student values:", "Seki" in student.values())
```

```
(.venv) C:\Users\MSI PC\OneDrive\Deskto
Python in subjects: True
English in subjects: False
English not in subjects: True
Technology in university: True
name in student: True
Seki in student: False
Seki in student values: True
```

- **Conditional statements:** used to control the flow of execution in a program based on specific conditions. They allow programs to execute different blocks of code depending on whether a condition evaluates to True or False.

```python
age = 19
category = "Adult" if age >= 18 else "Minor"
print("Category:", category)

a = 15
b = 20

larger = a if a > b else b
print("Larger number:", larger)

score = 85
grade = (
    "A" if score >= 90
    else "B" if score >= 80
    else "C" if score >= 70
    else "D" if score >= 50
    else "F"
)

print("Grade:", grade)
```

```
(.venv) C:\Users\MSI
Category: Adult
Larger number: 20
Grade: B
```

c. Precedence and Associativity of Operators

Operator precedence and associativity determine the order in which Python evaluates expressions containing multiple operators. Precedence decides which operator is evaluated first, while associativity determines the evaluation order when operators have the same precedence. Understanding these rules helps in writing correct and readable expressions.

Operator Precedence	Operator Associativity
Determines which operator is evaluated first.	Determines the evaluation order when operators have the same precedence.
Applies to operators with different precedence levels.	Applies only to operators with equal precedence.
Example: 10 + 20 * 30	Example: 100 / 10 * 10

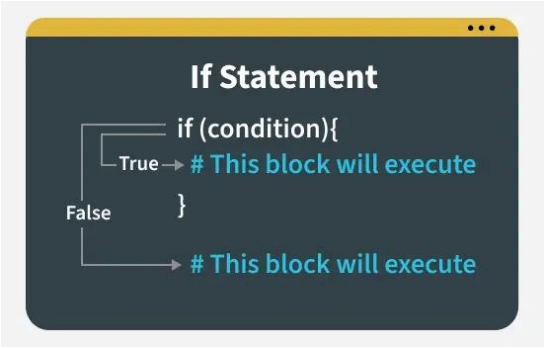




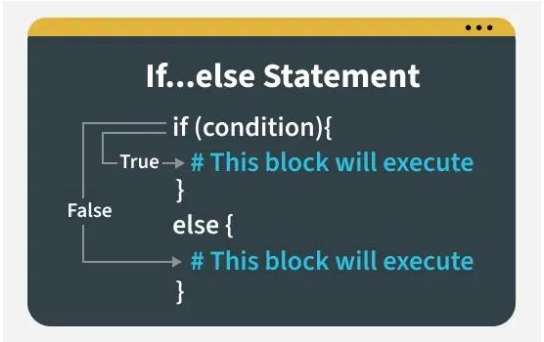
Chapter 7: Python Decision-Making Statements

I. Definition and Explanation

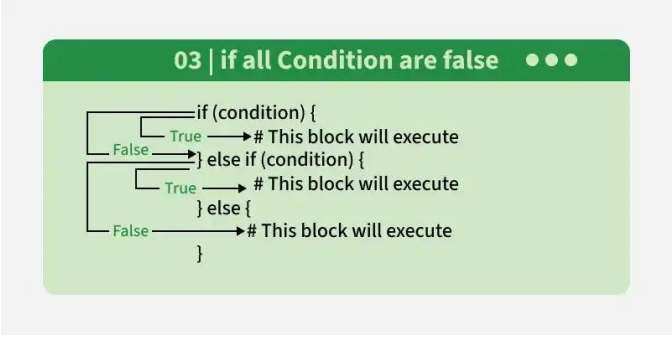
a. **The if Statement**: used to execute a block of code only when a given condition is True. If the condition is False, the code inside the if block is skipped.



b. if-Else Statement: used to execute one block of code when a condition is True and another block when the condition is False. It helps programs make decisions based on different conditions.



c. **If-elif-else Statement:** used to check multiple conditions in a program. It executes a block of code when its condition evaluates to True after previous conditions evaluate to False.



II. Employee Salary Management System in Python

```python
employee_name = "Seki"
basic_salary = 2500.00

# Calculate bonus based on basic salary
if basic_salary >= 3000:
    bonus_rate = 0.20
    salary_category = "High Salary"
elif basic_salary >= 2000:
    bonus_rate = 0.15
    salary_category = "Upper-Middle Salary"
elif basic_salary >= 1000:
    bonus_rate = 0.10
    salary_category = "Middle Salary"
elif basic_salary >= 500:
    bonus_rate = 0.05
    salary_category = "Entry-Level Salary"
else:
    bonus_rate = 0.00
    salary_category = "Low Salary"
```

```python
# Calculate bonus and gross salary
bonus = basic_salary * bonus_rate
gross_salary = basic_salary + bonus

# Calculate tax based on gross salary
if gross_salary >= 2000:
    tax_rate = 0.10
elif gross_salary >= 1000:
    tax_rate = 0.05
else:
    tax_rate = 0.00

# Calculate tax and net salary
tax = gross_salary * tax_rate
net_salary = gross_salary - tax

# Display the salary report
print("========== EMPLOYEE SALARY REPORT ==========")
print("Employee Name:", employee_name)
print("Basic Salary: $", round(basic_salary, 2))
print("Salary Category:", salary_category)
print("Bonus Rate:", bonus_rate * 100, "%")
print("Bonus Amount: $", round(bonus, 2))
print("Gross Salary: $", round(gross_salary, 2))
print("Tax Rate:", tax_rate * 100, "%")
print("Tax Deduction: $", round(tax, 2))
print("Net Salary: $", round(net_salary, 2))
print("============================================")
```

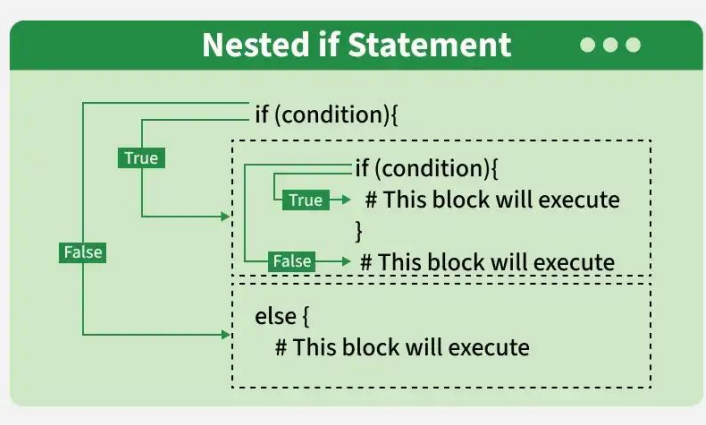
➤ Output

```
========== EMPLOYEE SALARY REPORT ==========
Employee Name: Seki
Basic Salary: $ 2500.0
Salary Category: Upper-Middle Salary
Bonus Rate: 15.0 %
Bonus Amount: $ 375.0
Gross Salary: $ 2875.0
Tax Rate: 10.0 %
Tax Deduction: $ 287.5
Net Salary: $ 2587.5
============================================
```

Chapter 8: Python Decision-Making - Part 2

I. Definition and Explanation

a. **Nested if Statements: an if statement** placed inside another if or else block. It is used when a decision depends on multiple conditions that need to be checked one after another.



b. **Multiple-Branch Selection with match-case (Python 3.10+) or if-elif-else:** used to perform pattern matching. It allows a value to be compared against multiple patterns and executes the code block associated with the first match. It provides a more organized alternative to long if-elif-else statements when handling multiple conditions.

```python
class Circle:
    __match_args__ = ("radius",)

    def __init__(self, radius):
        self.radius = radius

class Rectangle:
    __match_args__ = ("width", "height")

    def __init__(self, width, height):
        self.width = width
        self.height = height

def check_shape(shape):
    match shape:
        case Circle(radius):
            print(f"Circle radius: {radius}")
        case Rectangle(width, height):
            print(f"Rectangle: {width} x {height}")
        case _:
            print("Unknown shape")
```

II. Day of week example matching

```python
day = 3

match day:
    case 1:
        day_name = "Monday"
        message = "Start of the working week!"
    case 2:
        day_name = "Tuesday"
        message = "Keep working hard!"
    case 3:
        day_name = "Wednesday"
        message = "You are halfway through the week!"
    case 4:
        day_name = "Thursday"
        message = "Keep going! The weekend is near."
    case 5:
        day_name = "Friday"
        message = "Almost the weekend!"
    case 6:
        day_name = "Saturday"
        message = "Enjoy your weekend!"
    case 7:
        day_name = "Sunday"
        message = "Relax and prepare for next week!"
    case _:
        day_name = "Invalid day"
        message = "Please use a number from 1 to 7."

# Display the result
print("========== DAY OF THE WEEK ==========")
print("Day Number:", day)
print("Day Name:", day_name)
print("Message:", message)
print("====================================")
```

➢ Output

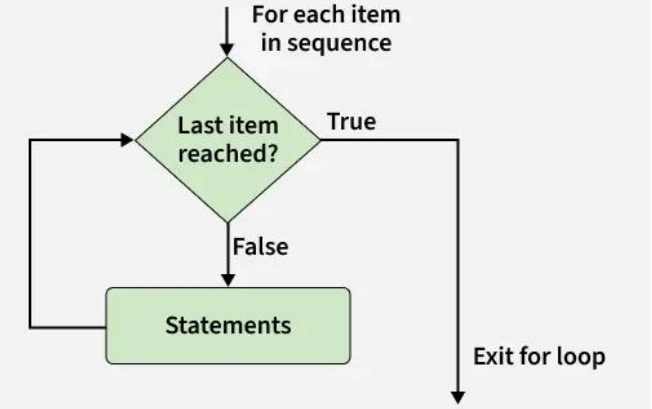
```
========== DAY OF THE WEEK ==========
Day Number: 3
Day Name: Wednesday
Message: You are halfway through the week!
====================================
```

Chapter 9: Python Loops and Loop Control

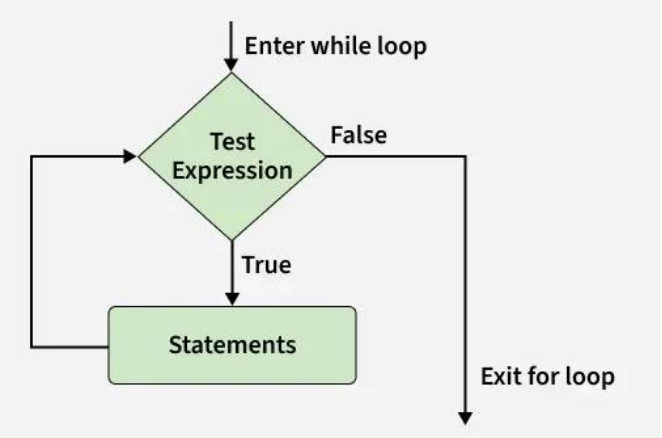
A **loop** allows you to execute a block of code repeatedly without writing the same code many times.

I. Definition and Explanation

a. **For loop**: used to iterate over a sequence such as a list, tuple, string or range. It executes a block of code once for each item in the sequence.



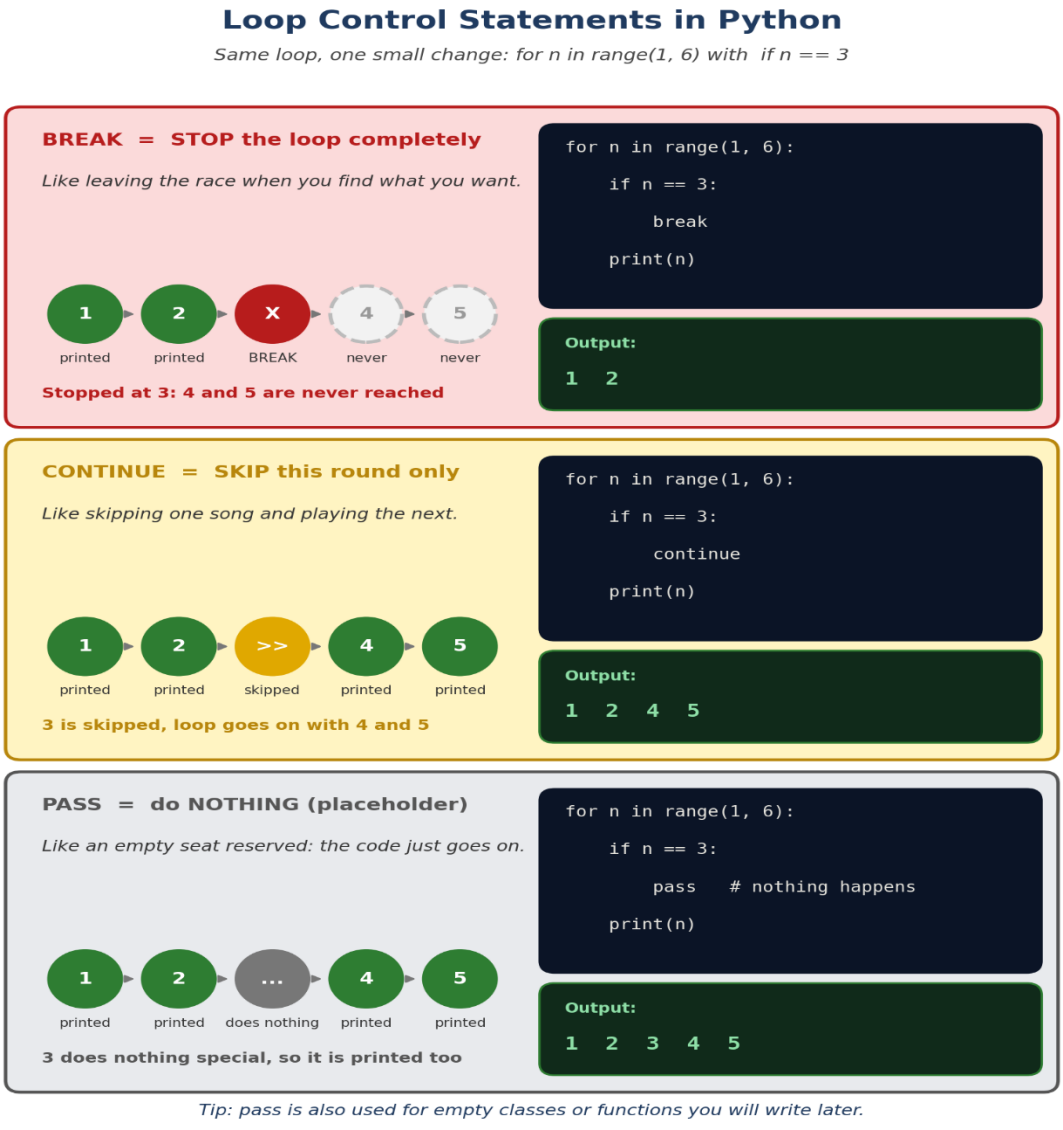
b. **While Loop:** used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.



c. **Loop Control Statements: break, continue and pass:**

- **Break:** is a statement in Python is used to exit or "break" out of a loop (either a for or while loop) prematurely, before the loop has iterated through all its items or reached its condition. When the break statement is executed, the program immediately exits the loop, and the control moves to the next line of code after the loop.

- **Continue Statement in Python:** a loop control statement that forces to execute the next iteration of the loop while skipping the rest of the code inside the loop for the current iteration only, i.e. when the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped for the current iteration and the next iteration of the loop will begin.
- **Pass**: used when a statement is syntactically required but we don't want to execute any code. It does nothing but allows us to maintain the structure of our program.



II. Student Score Analysis System Example

```python
scores = [85, 45, -5, 90, 0, 75, -1, 60]
total = 0
valid_count = 0
pass_count = 0

for score in scores:

    # BREAK: Stop the loop when the score is -1
    if score == -1:
        print("Stop signal detected. Ending the program.")
        break

    # CONTINUE: Skip invalid negative scores
    if score < 0:
        print("Invalid score:", score, "- Skipping...")
        continue

    # PASS: Placeholder for additional processing
    if score == 0:
        pass

    # Process valid scores
    print("Processing score:", score)

    total += score
    valid_count += 1
```

➢ Output

```
========== FINAL REPORT ==========
Valid scores: 5
Students passed: 3
Students failed: 2
Average score: 59.0
==================================
```

Resorce and References

1. W3schoolpython
2. Python Tutorial
3. pythonresources
4. Python-for-Data-Analysis